Code Reference: main.cjs

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	main.cjs
Purpose	Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.
Lines	453
Size	15,134 bytes
Talks to	builder frontend, local backend, public website runtime, portfolio catalog, Cloudflare/AI layer, GitHub/release layer
Functions	21
Variables/constants	42
API mentions	0
Signals	43

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	const fs = require("node:fs");
Line 4	const fsp = require("node:fs/promises");
Line 5	const net = require("node:net");
Line 6	const path = require("node:path");

Important implementation signals

Item	Explanation
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const { execFile } = require("node:child_process");
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: const fs = require("node:fs");
Compiler or simulator at line 4	Part of the Compile Code workspace or language/tool detection. Evidence: const fsp = require("node:fs/promises");
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: const net = require("node:net");
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: const path = require("node:path");
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: const { pathToFileURL } = require("node:url");
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: const { promisify } = require("node:util");
Process execution at line 13	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
Security or auth at line 27	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers"
Cloudflare or AI at line 38	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const directoryRefreshes = ["cloudflare"];
Security or auth at line 48	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Git command at line 53	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,

Item	Explanation
Git command at line 54	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 55	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 56	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 57	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 58	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe"
Installer at line 61	Windows setup, update, shortcut, or installation behavior. Evidence: function dispatchBuilderMenuAction(action) {
Installer at line 110	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({ type: "new-project" }) },
Installer at line 111	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target" }) },
Installer at line 113	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type: "save-draft" }) },
Installer at line 114	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () => dispatchBuilderMenuAction({ type: "apply-site" }) },
Rich text at line 127	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: { role: "paste" },
Installer at line 141	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview" }) },
Installer at line 142	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
Installer at line 154	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Open Preferences", accelerator: "CmdOrCtrl+,", click: () => dispatchBuilderMenuAction({ type: "preferences" }) },
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light" }) },
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark" }) }
Installer at line 163	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type: "builder-guide" }) },
Network request at line 164	Frontend-backend communication or public web/API calls. Evidence: { label: "GitHub Releases", click: () => shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
File write at line 190	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
File write at line 191	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.copyFile(source, target);
File write at line 196	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(target, { recursive: true });
File write at line 212	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
Git command at line 241	Repository state, publishing, branch checks, or release automation. Evidence: throw lastError || new Error("Git executable was not found.");
Git command at line 250	Repository state, publishing, branch checks, or release automation. Evidence: return fileExists(path.join(workspaceRoot, ".git"));
File write at line 267	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });
Installer at line 279	Windows setup, update, shortcut, or installation behavior. Evidence: const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
Installer at line 280	Windows setup, update, shortcut, or installation behavior. Evidence: ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
File write at line 286	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });
File write at line 287	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(portfolioRoot, { recursive: true });

Item	Explanation
Network request at line 363	Frontend-backend communication or public web/API calls. Evidence: return `http://127.0.0.1:${port}`;
Rich text at line 377	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: backgroundColor: "#eef8fd",

Important constants and variables

Item	Explanation
fs	Line 3: const fs = require("node:fs");
fsp	Line 4: const fsp = require("node:fs/promises");
net	Line 5: const net = require("node:net");
path	Line 6: const path = require("node:path");
mainWindow	Line 10: let mainWindow = null;
builderOrigin	Line 11: let builderOrigin = "";
updateShutdownStarted	Line 12: let updateShutdownStarted = false;
execFileAsync	Line 13: const execFileAsync = promisify(execFile);
defaultRepositoryUrl	Line 14: const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY || "";
defaultWorkspaceHomeName	Line 15: const defaultWorkspaceHomeName = "OMB Portfolio Builder";
rootFilesToRefresh	Line 17: const rootFilesToRefresh = [
rootFilesToSeed	Line 30: const rootFilesToSeed = [
directoryRefreshes	Line 38: const directoryRefreshes = ["cloudflare"];
directorySeeds	Line 39: const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];
portfolioFilesToSeed	Line 40: const portfolioFilesToSeed = [
portfolioDirectoriesToSeed	Line 51: const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];
gitCandidates	Line 52: const gitCandidates = [
payload	Line 63: const payload = JSON.stringify(action);
shouldFullscreen	Line 98: const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();
entries	Line 197: const entries = await fsp.readdir(source, { withFileTypes: true });
sourcePath	Line 199: const sourcePath = path.join(source, entry.name);
targetPath	Line 200: const targetPath = path.join(target, entry.name);
entries	Line 218: const entries = await fsp.readdir(directory);
lastError	Line 226: let lastError = null;
current	Line 254: let current = path.dirname(process.execPath);
next	Line 257: const next = path.dirname(current);
bundledSiteRoot	Line 278: const bundledSiteRoot = path.join(process.resourcesPath, "site");
defaultWorkspaceHome	Line 279: const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
workspaceHome	Line 282: const workspaceHome = process.env.OMB_WORKSPACE_HOME || defaultWorkspaceHome;
workspaceRoot	Line 283: const workspaceRoot = path.join(workspaceHome, "builder");
portfolioRoot	Line 284: const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE || path.join(workspaceHome, "portfolio");
envWorkspace	Line 315: const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;
workspaceRoot	Line 323: const workspaceRoot = path.resolve(__dirname);
nearbyRepository	Line 329: const nearbyRepository = findRepositoryNearExecutable();
tester	Line 341: const tester = net.createServer();

Item	Explanation
address	Line 344: const address = tester.address();
port	Line 345: const port = typeof address === "object" && address ? address.port : 0;
port	Line 352: const port = await findFreePort();
serverPath	Line 361: const serverPath = path.join(workspaceRoot, "server.mjs");
iconPath	Line 367: const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
refreshFullscreenMenu	Line 388: const refreshFullscreenMenu = () => {
gotLock	Line 426: const gotLock = app.requestSingleInstanceLock();

Function-by-function detail

dispatchBuilderMenuAction - lines 61-68

Item	Explanation
Source location	Lines 61-68 (8 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function dispatchBuilderMenuAction(action) {
Touches	filesystem
Calls detected	isDestroyed, stringify, executeJavaScript, dispatchEvent, CustomEvent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
61: function dispatchBuilderMenuAction(action) {
62:   if (!mainWindow || mainWindow.isDestroyed()) return;
63:   const payload = JSON.stringify(action);
64:   mainWindow.webContents.executeJavaScript(
65:     `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
66:     true
67:   ).catch(() => {});
68: }
```

quitForBuilderUpdate - lines 70-92

Item	Explanation
Source location	Lines 70-92 (23 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function quitForBuilderUpdate() {
Touches	Mostly local calculations or local UI state.
Calls detected	isDestroyed, destroy, quit, exit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
70: function quitForBuilderUpdate() {
71:   if (updateShutdownStarted) return;
72:   updateShutdownStarted = true;
```

```
  try {
    if (mainWindow && !mainWindow.isDestroyed()) {
      mainWindow.destroy();
    }
  } catch {
    // The process exit fallback below still guarantees the updater can continue.
  }
  try {
    app.quit();
  } catch {
    // Fall through to app.exit.
  }
  setTimeout(() => {
    try {
      app.exit(0);
    } catch {
      process.exit(0);
    }
  }, 1500);
}
```

setBuilderFullScreen - lines 96-103

Item	Explanation
Source location	Lines 96-103 (8 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function setBuilderFullScreen(nextState = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	isDestroyed, isFullScreen, setFullScreen, setMenuBarVisibility, setAutoHideMenuBar, setMenu, createAppMenu
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function setBuilderFullScreen(nextState = null) {
  if (!mainWindow || mainWindow.isDestroyed()) return;
  const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();
  mainWindow.setFullScreen(shouldFullscreen);
  mainWindow.setMenuBarVisibility(true);
  mainWindow.setAutoHideMenuBar(false);
  mainWindow.setMenu(createAppMenu(builderOrigin));
}
```

createAppMenu - lines 105-184

Item	Explanation
Source location	Lines 105-184 (80 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createAppMenu(origin) {
Touches	filesystem, network/API requests
Calls detected	buildFromTemplate, dispatchBuilderMenuAction, setBuilderFullScreen, openExternal, isDestroyed, isMinimized, restore, focus, fileExists, accessSync
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function createAppMenu(origin) {
  return Menu.buildFromTemplate([
    {
```

```
  108:       label: "File",
  109:       submenu: [
  110:         { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({
type: "new-project" }) },
  111:         { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target"
}) },
  112:         { type: "separator" },
  113:         { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type:
 "save-draft" }) },
  114:         { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () =>
dispatchBuilderMenuAction({ type: "apply-site" }) },
  115:         { type: "separator" },
  116:         { role: "quit", label: "Exit" }
  117:       ]
  118:     },
  119:     {
  120:       label: "Edit",
  121:       submenu: [
  122:         { role: "undo" },
  123:         { role: "redo" },
  124:         { type: "separator" },
  125:         { role: "cut" },
  126:         { role: "copy" },
  127:         { role: "paste" },
  128:         { role: "selectAll" }
  129:       ]
  130:     },
  131:     {
  132:       label: "View",
  133:       submenu: [
  134:         { role: "reload" },
  135:         { role: "forceReload" },
  136:         { type: "separator" },
  137:         { role: "resetZoom" },
  138:         { role: "zoomIn" },
  139:         { role: "zoomOut" },
  140:         { type: "separator" },
  141:         { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview"
}) },
  142:         { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
  143:         { type: "separator" },
  144:         {
  145:           label: mainWindow?.isFullScreen?.() ? "Exit Full Screen" : "Toggle Full Screen",
  146:           accelerator: "F11",
  147:           click: () => setBuilderFullScreen()
  148:         }
  149:       ]
  150:     },
  151:     {
  152:       label: "Preferences",
  153:       submenu: [
  154:         { label: "Open Preferences", accelerator: "CmdOrCtrl+,", click: () => dispatchBuilderMenuAction({
 type: "preferences" }) },
  155:         { type: "separator" },
  156:         { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light"
 }) },
  157:         { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark"
}) }
  158:       ]
  159:     },
  160:     {
  161:       label: "Help",
  162:       submenu: [
  163:         { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type:
"builder-guide" }) },
  164:         { label: "GitHub Releases", click: () =>
shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
  165:         {
  166:           label: "Focus Builder Window",
  167:           click: () => {
  168:             if (!mainWindow || mainWindow.isDestroyed()) return;
  169:             if (mainWindow.isMinimized()) mainWindow.restore();
  170:             mainWindow.focus();
  171:           }
  172:         }
  173:       ]
  174:     }
  175:   ]);
  176: }
  177:
  178: function fileExists(filePath) {
  179:   try {
  180:     fs.accessSync(filePath);
  181:     return true;
  182:   } catch {
  183:     return false;
  184:   }
```

fileExists - lines 178-185

Item	Explanation
Source location	Lines 178-185 (8 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function fileExists(filePath) {
Touches	filesystem
Calls detected	accessSync
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
178: function fileExists(filePath) {
179:   try {
180:     fs.accessSync(filePath);
181:     return true;
182:   } catch {
183:     return false;
184:   }
185: }
```

copyFileIfAvailable - lines 187-192

Item	Explanation
Source location	Lines 187-192 (6 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function copyFileIfAvailable(source, target, overwrite = false) {
Touches	filesystem
Calls detected	fileExists, mkdir, dirname, copyFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
187: async function copyFileIfAvailable(source, target, overwrite = false) {
188:   if (!fileExists(source)) return;
189:   if (!overwrite && fileExists(target)) return;
190:   await fsp.mkdir(path.dirname(target), { recursive: true });
191:   await fsp.copyFile(source, target);
192: }
```

copyDirectoryMissingFiles - lines 194-207

Item	Explanation
Source location	Lines 194-207 (14 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function copyDirectoryMissingFiles(source, target) {
Touches	filesystem
Calls detected	fileExists, mkdir, readdir, join, isDirectory, isFile, copyFileIfAvailable
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	4 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
194: async function copyDirectoryMissingFiles(source, target) {
195:   if (!fileExists(source)) return;
196:   await fsp.mkdir(target, { recursive: true });
197:   const entries = await fsp.readdir(source, { withFileTypes: true });
198:   for (const entry of entries) {
199:     const sourcePath = path.join(source, entry.name);
200:     const targetPath = path.join(target, entry.name);
201:     if (entry.isDirectory()) {
202:       await copyDirectoryMissingFiles(sourcePath, targetPath);
203:     } else if (entry.isFile()) {
204:       await copyFileIfAvailable(sourcePath, targetPath, false);
205:     }
206:   }
207: }
```

copyDirectoryRefresh - lines 209-214

Item	Explanation
Source location	Lines 209-214 (6 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function copyDirectoryRefresh(source, target) {
Touches	filesystem
Calls detected	fileExists, rm, mkdir, dirname, cp
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
209: async function copyDirectoryRefresh(source, target) {
210:   if (!fileExists(source)) return;
211:   await fsp.rm(target, { recursive: true, force: true });
212:   await fsp.mkdir(path.dirname(target), { recursive: true });
213:   await fsp.cp(source, target, { recursive: true, force: true });
214: }
```

directoryIsEmpty - lines 216-223

Item	Explanation
Source location	Lines 216-223 (8 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function directoryIsEmpty(directory) {
Touches	Mostly local calculations or local UI state.
Calls detected	readdir
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
216: async function directoryIsEmpty(directory) {
```

```
217:   try {
218:     const entries = await fsp.readdir(directory);
219:     return entries.length === 0;
220:   } catch {
221:     return true;
222:   }
223: }
```

runGit - lines 225-242

Item	Explanation
Source location	Lines 225-242 (18 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function runGit(args, cwd) {
Touches	Mostly local calculations or local UI state.
Calls detected	execFileAsync, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
225: async function runGit(args, cwd) {
226:   let lastError = null;
227:   for (const candidate of gitCandidates) {
228:     try {
229:       await execFileAsync(candidate, args, {
230:         cwd,
231:         maxBuffer: 10 * 1024 * 1024,
232:         windowsHide: true
233:       });
234:       return true;
235:     } catch (error) {
236:       lastError = error;
237:       if (error.code === "ENOENT") continue;
238:       throw error;
239:     }
240:   }
241:   throw lastError || new Error("Git executable was not found.");
242: }
```

workspaceLooksUsable - lines 244-247

Item	Explanation
Source location	Lines 244-247 (4 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function workspaceLooksUsable(workspaceRoot) {
Touches	Mostly local calculations or local UI state.
Calls detected	fileExists, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
244: function workspaceLooksUsable(workspaceRoot) {
245:   return fileExists(path.join(workspaceRoot, "server.mjs"))
246:     && fileExists(path.join(workspaceRoot, "index.html"));
247: }
```

workspaceIsGitBacked - lines 249-251

Item	Explanation
Source location	Lines 249-251 (3 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function workspaceIsGitBacked(workspaceRoot) {
Touches	Mostly local calculations or local UI state.
Calls detected	fileExists, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
249: function workspaceIsGitBacked(workspaceRoot) {
250:   return fileExists(path.join(workspaceRoot, ".git"));
251: }
```

findRepositoryNearExecutable - lines 253-262

Item	Explanation
Source location	Lines 253-262 (10 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function findRepositoryNearExecutable() {
Touches	Mostly local calculations or local UI state.
Calls detected	dirname, workspaceLooksUsable, workspaceIsGitBacked
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
253: function findRepositoryNearExecutable() {
254:   let current = path.dirname(process.execPath);
255:   for (let index = 0; index < 8; index += 1) {
256:     if (workspaceLooksUsable(current) && workspaceIsGitBacked(current)) return current;
257:     const next = path.dirname(current);
258:     if (next === current) break;
259:     current = next;
260:   }
261:   return "";
262: }
```

cloneWorkspaceIfPossible - lines 264-275

Item	Explanation
Source location	Lines 264-275 (12 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function cloneWorkspaceIfPossible(workspaceRoot) {
Touches	filesystem
Calls detected	workspaceIsGitBacked, mkdir, directoryIsEmpty, runGit, dirname
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  264: async function cloneWorkspaceIfPossible(workspaceRoot) {
  265:   if (!defaultRepositoryUrl) return false;
  266:   if (workspaceIsGitBacked(workspaceRoot)) return true;
  267:   await fsp.mkdir(workspaceRoot, { recursive: true });
  268:   if (!(await directoryIsEmpty(workspaceRoot))) return false;
  269:   try {
  270:     await runGit(["clone", "--depth", "1", defaultRepositoryUrl, workspaceRoot],
path.dirname(workspaceRoot));
  271:     return workspaceIsGitBacked(workspaceRoot);
  272:   } catch {
  273:     return false;
  274:   }
  275: }
```

preparePackagedWorkspace - lines 277-312

Item	Explanation
Source location	Lines 277-312 (36 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function preparePackagedWorkspace() {
Touches	filesystem
Calls detected	join, getPath, mkdir, cloneWorkspaceIfPossible, copyFileIfAvailable, copyDirectoryRefresh, copyDirectoryMissingFiles
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	10 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  277: async function preparePackagedWorkspace() {
  278:   const bundledSiteRoot = path.join(process.resourcesPath, "site");
  279:   const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
  280:     ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
  281:     : path.join(app.getPath("userData"), "workspace");
  282:   const workspaceHome = process.env.OMB_WORKSPACE_HOME || defaultWorkspaceHome;
  283:   const workspaceRoot = path.join(workspaceHome, "builder");
  284:   const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE || path.join(workspaceHome, "portfolio");
  285:
  286:   await fsp.mkdir(workspaceRoot, { recursive: true });
  287:   await fsp.mkdir(portfolioRoot, { recursive: true });
  288:   await cloneWorkspaceIfPossible(portfolioRoot);
  289:
  290:   for (const fileName of rootFilesToRefresh) {
  291:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
true);
  292:   }
  293:   for (const fileName of rootFilesToSeed) {
  294:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
false);
  295:   }
  296:   for (const directoryName of directoryRefreshes) {
  297:     await copyDirectoryRefresh(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
  298:   }
  299:   for (const directoryName of directorySeeds) {
  300:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
  301:   }
  302:
  303:   for (const fileName of portfolioFilesToSeed) {
  304:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(portfolioRoot, fileName),
false);
  305:   }
  306:   await copyFileIfAvailable(path.join(bundledSiteRoot, "portfolio-README.md"), path.join(portfolioRoot,
"README.md"), false);
```

```
  307:   for (const directoryName of portfolioDirectoriesToSeed) {
  308:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(portfolioRoot,
directoryName));
  309:   }
  310:
  311:   return { workspaceRoot, portfolioRoot };
  312: }
```

resolveWorkspaceRoots - lines 314-337

Item	Explanation
Source location	Lines 314-337 (24 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function resolveWorkspaceRoots() {
Touches	Mostly local calculations or local UI state.
Calls detected	workspaceLooksUsable, join, dirname, resolve, findRepositoryNearExecutable, preparePackagedWorkspace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  314: async function resolveWorkspaceRoots() {
  315:   const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;
  316:   if (envWorkspace && workspaceLooksUsable(envWorkspace)) {
  317:     return {
  318:       workspaceRoot: envWorkspace,
  319:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || path.join(path.dirname(envWorkspace),
"portfolio")
  320:     };
  321:   }
  322:   if (!app.isPackaged) {
  323:     const workspaceRoot = path.resolve(__dirname);
  324:     return {
  325:       workspaceRoot,
  326:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || workspaceRoot
  327:     };
  328:   }
  329:   const nearbyRepository = findRepositoryNearExecutable();
  330:   if (nearbyRepository) {
  331:     return {
  332:       workspaceRoot: nearbyRepository,
  333:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || nearbyRepository
  334:     };
  335:   }
  336:   return preparePackagedWorkspace();
  337: }
```

findFreePort - lines 339-349

Item	Explanation
Source location	Lines 339-349 (11 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	function findFreePort() {
Touches	Mostly local calculations or local UI state.
Calls detected	createServer, once, listen, address, close, resolve
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
339: function findFreePort() {
340:   return new Promise((resolve, reject) => {
341:     const tester = net.createServer();
342:     tester.once("error", reject);
343:     tester.listen(0, "127.0.0.1", () => {
344:       const address = tester.address();
345:       const port = typeof address === "object" && address ? address.port : 0;
346:       tester.close(() => resolve(port));
347:     });
348:   });
349: }
```

startBuilderServer - lines 351-364

Item	Explanation
Source location	Lines 351-364 (14 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function startBuilderServer(workspaceRoot, portfolioRoot) {
Touches	network/API requests
Calls detected	findFreePort, getVersion, join, pathToFileURL, now
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
351: async function startBuilderServer(workspaceRoot, portfolioRoot) {
352:   const port = await findFreePort();
353:   process.env.PORT = String(port);
354:   process.env.HOST = "127.0.0.1";
355:   process.env.OMB_DESKTOP_APP = "1";
356:   process.env.OMB_BUILDER_WORKSPACE = workspaceRoot;
357:   process.env.OMB_PORTFOLIO_WORKSPACE = portfolioRoot;
358:   process.env.OMB_BUILDER_REPOSITORY = defaultRepositoryUrl;
359:   process.env.OMB_APP_VERSION = app.getVersion();
360:
361:   const serverPath = path.join(workspaceRoot, "server.mjs");
362:   await import(`${pathToFileURL(serverPath).href}?desktop=${Date.now()}`);
363:   return `http://127.0.0.1:${port}`;
364: }
```

createWindow - lines 366-423

Item	Explanation
Source location	Lines 366-423 (58 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createWindow(workspaceRoot, origin) {
Touches	Mostly local calculations or local UI state.
Calls detected	join, BrowserWindow, fileExists, setMenu, createAppMenu, setMenuBarVisibility, setAutoHideMenuBar, isDestroyed, on, preventDefault, setBuilderFullScreen, isFullScreen
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
366: function createWindow(workspaceRoot, origin) {
367:   const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
368:   nativeTheme.themeSource = "light";
369:   mainWindow = new BrowserWindow({
370:     width: 1440,
371:     height: 960,
```

```
372:     minWidth: 980,
373:     minHeight: 680,
374:     title: "OMB Portfolio Builder",
375:     autoHideMenuBar: false,
376:     icon: fileExists(iconPath) ? iconPath : undefined,
377:     backgroundColor: "#eef8fd",
378:     webPreferences: {
379:       contextIsolation: true,
380:       nodeIntegration: false,
381:       sandbox: true
382:     }
383:   });
384:   mainWindow.setMenu(createAppMenu(origin));
385:   mainWindow.setMenuBarVisibility(true);
386:   mainWindow.setAutoHideMenuBar(false);
387:
388:   const refreshFullscreenMenu = () => {
389:     if (mainWindow && !mainWindow.isDestroyed()) {
390:       mainWindow.setMenuBarVisibility(true);
391:       mainWindow.setAutoHideMenuBar(false);
392:       mainWindow.setMenu(createAppMenu(origin));
393:     }
394:   };
395:   mainWindow.on("enter-full-screen", refreshFullscreenMenu);
396:   mainWindow.on("leave-full-screen", refreshFullscreenMenu);
397:
398:   mainWindow.webContents.on("before-input-event", (event, input) => {
399:     if (input.key === "F11") {
400:       event.preventDefault();
401:       setBuilderFullScreen();
402:       return;
403:     }
404:     if (input.key === "Escape" && mainWindow.isFullScreen()) {
405:       event.preventDefault();
406:       setBuilderFullScreen(false);
407:     }
408:   });
409:
410:   mainWindow.webContents.setWindowOpenHandler(({ url }) => {
411:     if (url.startsWith(origin)) return { action: "allow" };
412:     shell.openExternal(url);
413:     return { action: "deny" };
414:   });
415:
416:   mainWindow.webContents.on("will-navigate", (event, url) => {
417:     if (url.startsWith(origin)) return;
418:     event.preventDefault();
419:     shell.openExternal(url);
420:   });
421:
422:   mainWindow.loadURL(`${origin}/template-preview.html`);
423: }
```

refreshFullscreenMenu - lines 388-394

Item	Explanation
Source location	Lines 388-394 (7 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	const refreshFullscreenMenu = () => {
Touches	Mostly local calculations or local UI state.
Calls detected	isDestroyed, setMenuBarVisibility, setAutoHideMenuBar, setMenu, createAppMenu
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
388:   const refreshFullscreenMenu = () => {
389:     if (mainWindow && !mainWindow.isDestroyed()) {
390:       mainWindow.setMenuBarVisibility(true);
391:       mainWindow.setAutoHideMenuBar(false);
392:       mainWindow.setMenu(createAppMenu(origin));
393:     }
394:   };
```

boot - lines 425-447

Item	Explanation
Source location	Lines 425-447 (23 source lines).
Purpose	Part of Electron desktop startup, window control, workspace setup, menu behavior, or update handoff.
Declaration	async function boot() {
Touches	Mostly local calculations or local UI state.
Calls detected	requestSingleInstanceLock, quit, on, isMinimized, restore, focus, whenReady, resolveWorkspaceRoots, startBuilderServer, createWindow, showErrorBox
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
425: async function boot() {
426:   const gotLock = app.requestSingleInstanceLock();
427:   if (!gotLock) {
428:     app.quit();
429:     return;
430:   }
431:
432:   app.on("second-instance", () => {
433:     if (!mainWindow) return;
434:     if (mainWindow.isMinimized()) mainWindow.restore();
435:     mainWindow.focus();
436:   });
437:
438:   await app.whenReady();
439:   try {
440:     const { workspaceRoot, portfolioRoot } = await resolveWorkspaceRoots();
441:     builderOrigin = await startBuilderServer(workspaceRoot, portfolioRoot);
442:     createWindow(workspaceRoot, builderOrigin);
443:   } catch (error) {
444:     dialog.showErrorBox("OMB Portfolio Builder could not start", error?.message || String(error));
445:     app.quit();
446:   }
447: }
```

Representative opening snippet

```
 1: const { app, BrowserWindow, Menu, dialog, nativeTheme, shell } = require("electron");
 2: const { execFile } = require("node:child_process");
 3: const fs = require("node:fs");
 4: const fsp = require("node:fs/promises");
 5: const net = require("node:net");
 6: const path = require("node:path");
 7: const { pathToFileURL } = require("node:url");
 8: const { promisify } = require("node:util");
 9:
10: let mainWindow = null;
11: let builderOrigin = "";
12: let updateShutdownStarted = false;
13: const execFileAsync = promisify(execFile);
14: const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY || "";
15: const defaultWorkspaceHomeName = "OMB Portfolio Builder";
16:
17: const rootFilesToRefresh = [
18:   "server.mjs",
19:   "index.html",
20:   "styles.css",
21:   "script.js",
22:   "electronics-search.js",
23:   "builder-rich-future-sections.js",
24:   "template-preview.html",
25:   "template-preview.css",
26:   "template-preview.js",
27:   "_headers"
28: ];
29:
30: const rootFilesToSeed = [
31:   "projects.json",
32:   "project-templates.json",
33:   "README.md",
34:   ".nojekyll",
35:   "robots.txt"
```

```
];

const directoryRefreshes = ["cloudflare"];
const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];
const portfolioFilesToSeed = [
  "projects.json",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  ".nojekyll",
  "robots.txt",
  "_headers",
  "sitemap.xml"
];
const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];
const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe"
].filter(Boolean);

function dispatchBuilderMenuAction(action) {
  if (!mainWindow || mainWindow.isDestroyed()) return;
  const payload = JSON.stringify(action);
  mainWindow.webContents.executeJavaScript(
    `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
    true
  ).catch(() => {});
}

function quitForBuilderUpdate() {
  if (updateShutdownStarted) return;
  updateShutdownStarted = true;
```